

EJERCICIO DE REPASO DE HERENCIA MULTIPLE (TEMA 2)

Crea una clase Abuelo que tenga 2 hijos (Padre y Madre)

Crea una clase Nieto que herede de Padre y Madre

Abuelo tiene un int x

Padre tiene un const int y, char *nombre;

Madre tiene un float f;

Nieto tiene un const int h y char *apodo;

Suponiendo que no puede haber constructores sin parámetros en ninguna de las tres clases, indica como serian la herencia, los constructores y destructores para que:

a) la clase Nieto herede Abuelo tanto de Padre como de Madre

b) la clase Nieto solo herede Abuelo una vez

Crea un método ver() en todas las clases. Cada uno de ellos debe mostrar todo lo que contiene (suyo propio o heredado) llamando en su interior a todos sus ver() ascendientes (abuelo, padre o madre).

Cuando Nieto herede Abuelo dos veces (via Padre y Madre), su método ver() debe en su interior llamar al ver() del Abuelo por vía paterna y materna.

Comprueba en el caso a) y b) si la clase Nieta pueda invocar en el main(), además de su propio método ver(), los métodos ver() que hereda de sus ancestros (Padre, Madre y Abuelo)

SOLUCION:

a) queremos que la clase Nieto herede Abuelo tanto de Padre como de Madre.

```
#include <iostream>
#include <cstring>

using namespace std;

class Abuelo {
    int x;
public:
    Abuelo(int i) { x=i; }
    virtual void ver() const { cout << "abuelo: " << x << endl; }
};

class Padre: public Abuelo {
    const int y;
    char *nombre;
public:
    Padre(int a, int b, char *nom): Abuelo(a), y(b) {
        nombre=new char [strlen(nom)+1]; strcpy(nombre, nom);
    }
    Padre(const Padre &c): Abuelo(c), y(c.y) {
        nombre=new char [strlen(c.nombre)+1]; strcpy(nombre, c.nombre);
    }
    virtual ~Padre() { delete [] nombre; }
    virtual void ver() const { Abuelo::ver(); cout << "padre: " << y << " " << nombre << endl; }
};

class Madre: public Abuelo {
    float f;
public:
    Madre(int i, float f): Abuelo(i) { this->f=f; }
    virtual void ver() const { Abuelo::ver(); cout << "madre: " << f << endl; }
};

class Nieto: public Padre, public Madre {
    const int h;
    char *apodo;
public:
    Nieto(int a, int b, char *nom, float f, int h1, char *mote): Padre(a,b,nom), Madre(a,f), h(h1) {
        apodo=new char [strlen(mote)+1]; strcpy(apodo, mote);
    }
    virtual ~Nieto() { delete [] apodo; }
    Nieto(const Nieto &n): Padre(n), Madre(n), h(n.h) {
        apodo=new char [strlen(n.apodo)+1]; strcpy(apodo, n.apodo);
    }
    virtual void ver() const {
        ((Padre*)this).Abuelo::ver(); //convierto objeto Nieto (*this) en objeto Padre para invocar ver de abuelo
        ((Madre*)this)->Abuelo::ver(); //convierto puntero Nieto (this) en puntero Madre para invocar ver de abuelo
        //((Abuelo*)this).ver(); //ERROR: Abuelo es una ambigua clase base de Nieto
        //((Abuelo*)this)->ver(); //ERROR: Abuelo es una ambigua clase base de Nieto
        //Abuelo::ver(); //ERROR ambigüedad Abuelo::ver() lo hereda 2 veces por parte de Padre y Madre
        Padre::ver(); Madre::ver(); cout << "nieto: " << h << " " << apodo << endl;
    }
};

int main() { //indico como llamarlo usando objetos (nieto) y punteros (&nieto)
    Nieto nieto(90,50,"Paco",45.8,20,"yo");
    nieto.ver(); cout << " - \n"; (&nieto)->ver(); cout << endl;
    nieto.Padre::ver(); cout << " - \n"; (&nieto)->Padre::ver(); cout << endl;
    nieto.Madre::ver(); cout << " - \n"; (&nieto)->Madre::ver(); cout << endl;
    //nieto.Abuelo::ver(); //ERROR ambigüedad Abuelo::ver() lo hereda 2 veces por parte de Padre y Madre

    //convierto objeto Nieto (nieto) en objeto Padre y puntero Nieto (&nieto) en puntero Padre
    ((Padre)nieto).Abuelo::ver(); cout << " - \n"; ((Padre *)&nieto)->Abuelo::ver(); cout << endl;
    //con Madre lo voy a hacer usando una variable local y puntero local tipo Madre;
    Madre m=nieto;
    Madre *p=&nieto;
    m.Abuelo::ver(); cout << " - \n"; p->Abuelo::ver(); cout << endl; //idem con Madre
    //((Abuelo)nieto).ver(); cout << endl; //ERROR: Abuelo es una ambigua clase base de Nieto
    //((Abuelo*)&nieto)->ver(); cout << endl; //ERROR: Abuelo es una ambigua clase base de Nieto
}
```

SALIDA POR PANTALLA

```
abuelo: 90
abuelo: 90
abuelo: 90
padre: 50 Paco
abuelo: 90
madre: 45.8
nieto: 20 yo
-
abuelo: 90
abuelo: 90
abuelo: 90
padre: 50 Paco
abuelo: 90
madre: 45.8
nieto: 20 yo
-
abuelo: 90
padre: 50 Paco
-
abuelo: 90
madre: 45.8
-
abuelo: 90
madre: 45.8
-
abuelo: 90
abuelo: 90
-
abuelo: 90
-
abuelo: 90
```

b) queremos que la clase Nieto solo herede Abuelo una vez

Para que Nieto herede Abuelo una sola vez tenemos que declarar la herencia **virtual en sus padres (hay que declarar la clase común como **virtual** en las clases que lo heredan).**

Para evitar que la clase Abuelo puede heredarse varias veces indirectamente en Nieto, hay que hacer que la herencia de la clase Abuelo sea **virtual** en Padre y Madre, que son las clases que hereda Nieto.

MUY IMPORTANTE!!!!

El constructor de la clase que hereda de clases con herencia **virtual debe invocar (además de a los constructores de sus clases padres) al constructor de la clase virtual heredada por sus padres.**

```
#include <iostream>
#include <cstring>
using namespace std;

class Abuelo {
    int x;
public:
    Abuelo(int i) { x=i; }
    virtual void ver() const { cout << "abuelo: " << x << endl; }
};

class Padre: virtual public Abuelo {
    const int y;
    char *nombre;
public:
    Padre(int a, int b, char *nom): Abuelo(a), y(b) {
        nombre=new char [strlen(nom)+1];
        strcpy(nombre, nom);
    }
    Padre(const Padre &c): Abuelo(c), y(c.y) {
        nombre=new char [strlen(c.nombre)+1];
        strcpy(nombre, c.nombre);
    }
    virtual ~Padre() { delete [] nombre; }
    virtual void ver() const { Abuelo::ver(); cout << "padre: " << y << " " << nombre << endl; }
};

class Madre: virtual public Abuelo {
    float f;
public:
    Madre(int i, float f): Abuelo(i) { this->f=f; }
    virtual void ver() const { Abuelo::ver(); cout << "madre: " << f << endl; }
};

class Nieto: public Padre, public Madre {
    const int h;
    char *apodo;
public:
    Nieto(int a, int b, char *nom, float f, int h1, char *mote): Abuelo(a), Padre(a,b,nom), Madre(a,f), h(h1) {
        apodo=new char [strlen(mote)+1];
        strcpy(apodo, mote);
    }
    virtual ~Nieto() { delete [] apodo; }
    Nieto(const Nieto &n): Abuelo(n), Padre(n), Madre(n), h(n.h) {
        apodo=new char [strlen(n.apodo)+1];
        strcpy(apodo, n.apodo);
    }
    virtual void ver() const {
        Abuelo::ver(); Padre::ver(); Madre::ver();
        cout << "nieto: " << h << " " << apodo << endl;
    }
};

int main() {
    Nieto nieto(90,50,"Paco",45.8,20,"yo");
    nieto.ver(); cout << endl;
    nieto.Padre::ver(); cout << endl;
    nieto.Madre::ver(); cout << endl;
    nieto.Abuelo::ver(); //no hay ambigüedad porque solo se hereda una vez
}
```

SALIDA POR PANTALLA

```
abuelo: 90
abuelo: 90
padre: 50 Paco
abuelo: 90
madre: 45.8
nieto: 20 yo
```

```
abuelo: 90
padre: 50 Paco
```

```
abuelo: 90
madre: 45.8
```

```
abuelo: 90
```